Java Programming (314317)

Inheritance

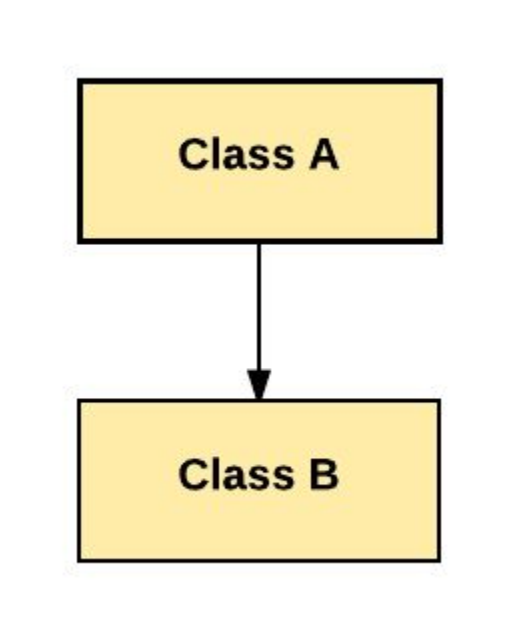
- Process of creating one class from existing class by acquiring properties and behavior of existing class.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

Inheritance

- Types of Inheritance
  - Single Inheritance
  - Multilevel Inheritance
  - Hierarchical Inheritance
  - Hybrid Inheritance
  - Multiple Inheritance

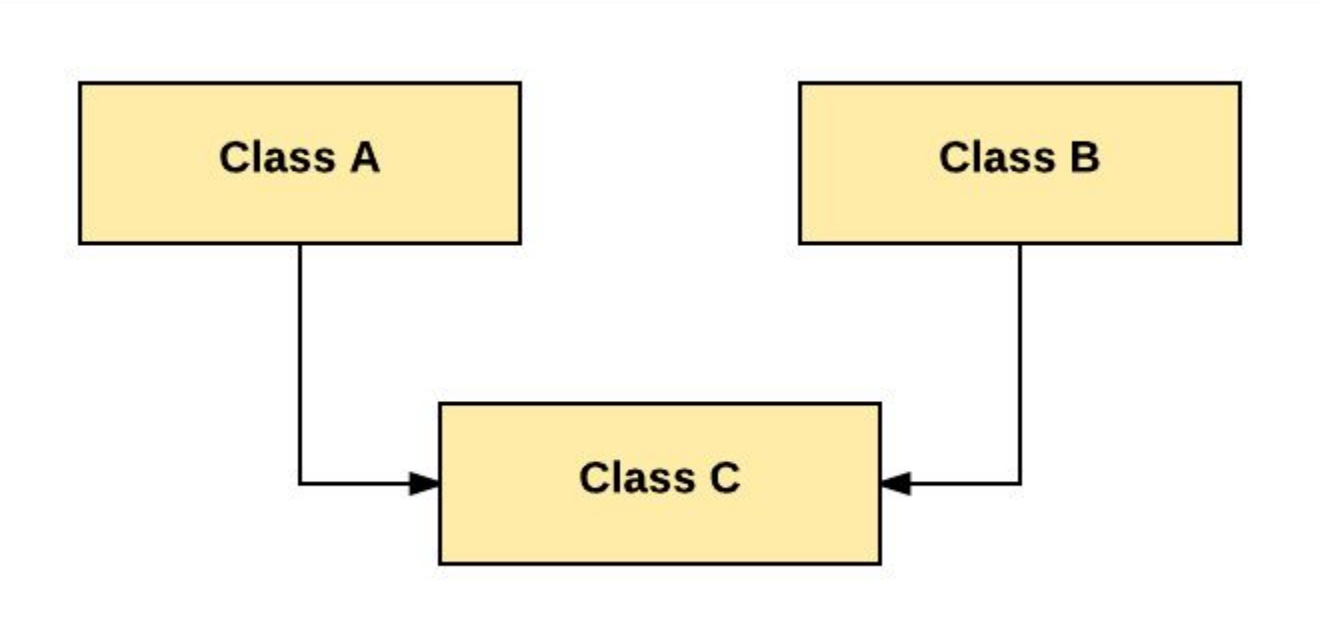
Single Inheritance:

- In Single Inheritance one class extends another class (one class only).



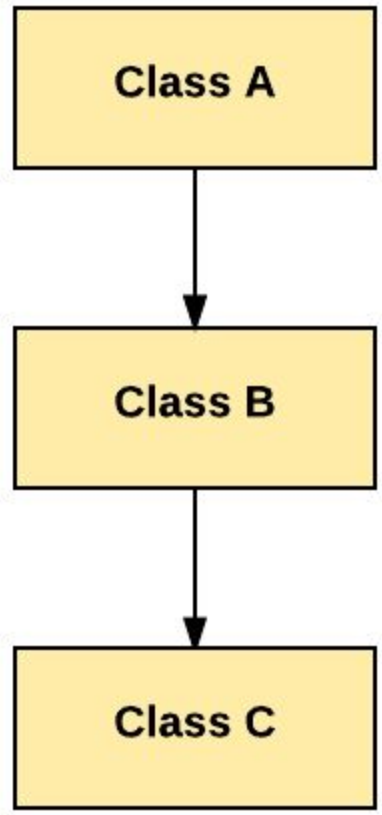
Multiple Inheritance:

- In Multiple Inheritance, one class extending more than one class. Java does not support multiple inheritance.



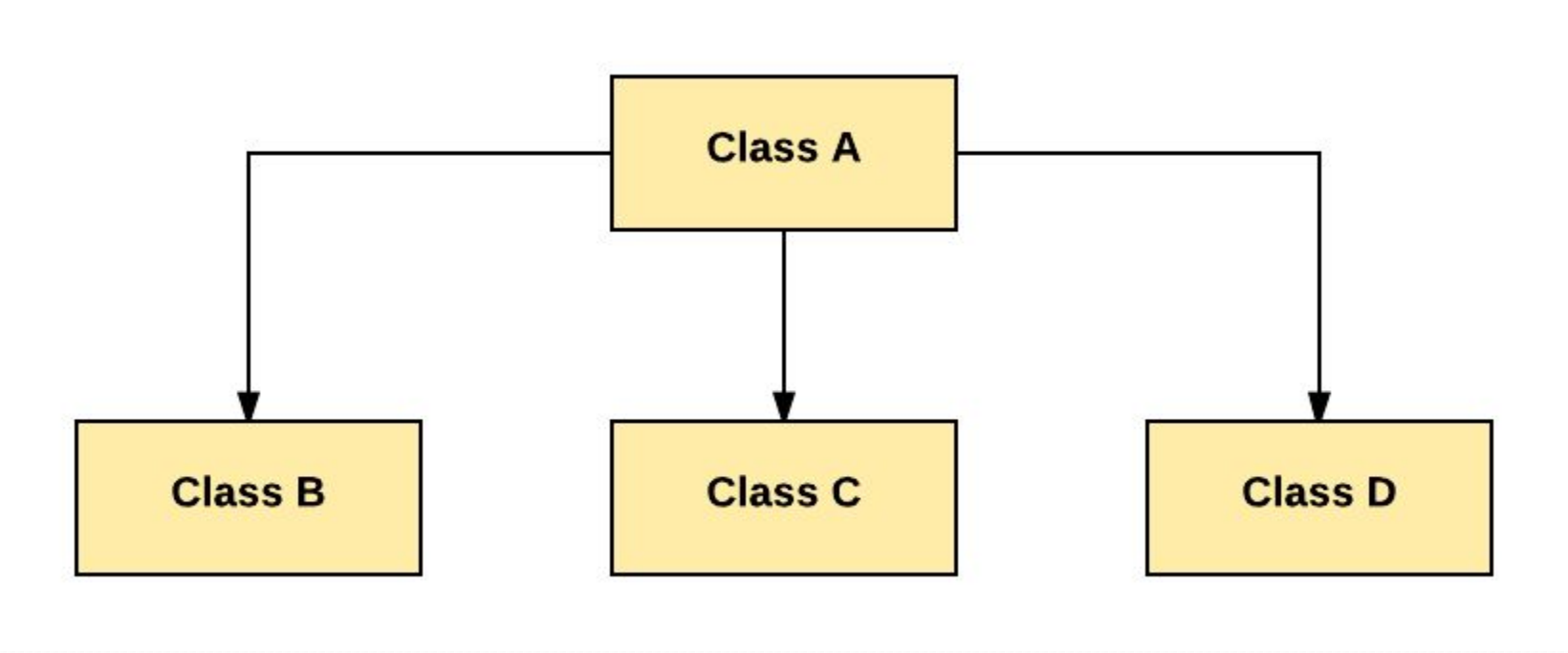
Multilevel Inheritance:

- In Multilevel Inheritance, one class can inherit from a derived class. Hence, the derived class becomes the base class for the new class.



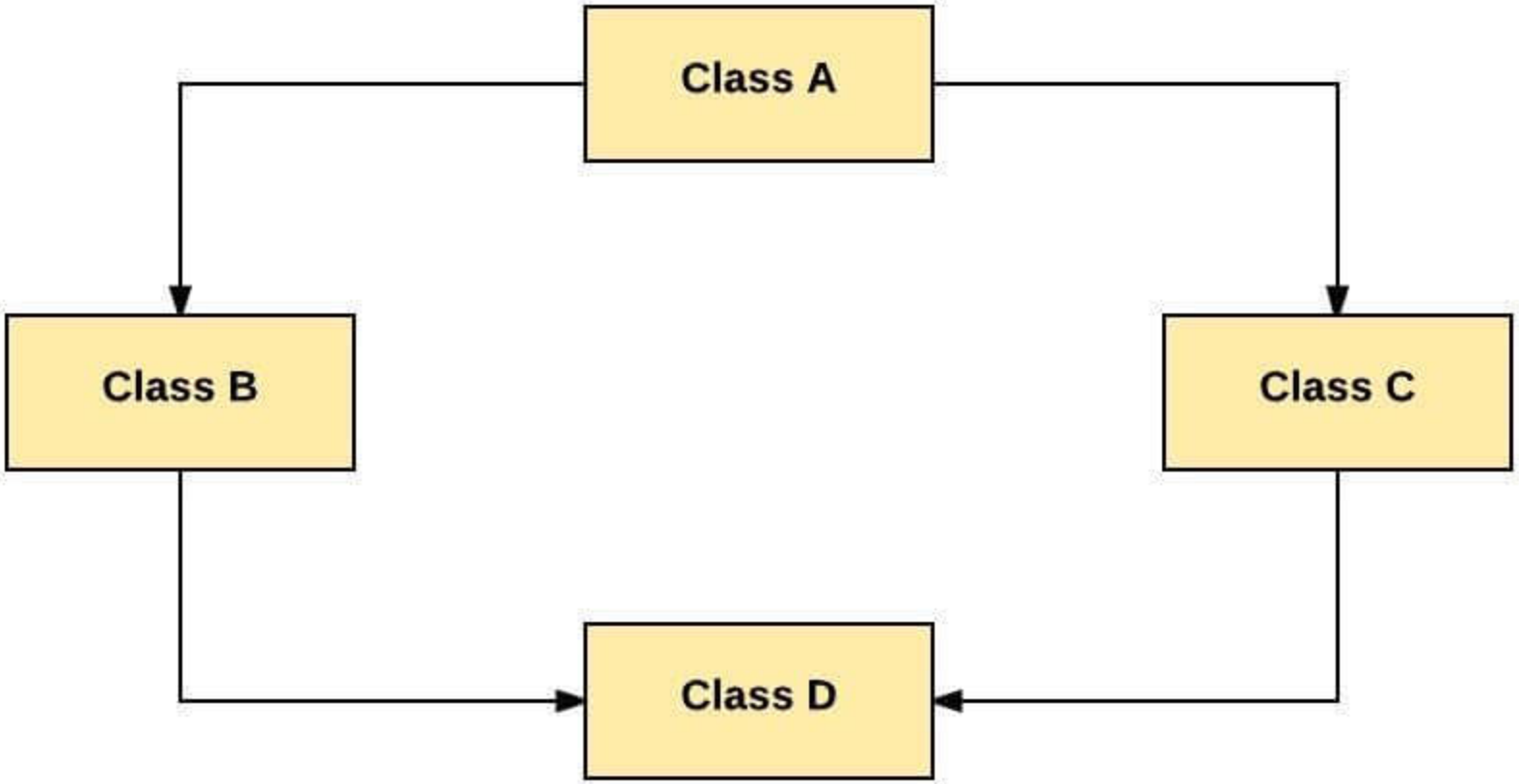
Hierarchical Inheritance:

- In Hierarchical Inheritance, one class is inherited by many sub classes.



Hybrid Inheritance:

- Hybrid inheritance is a combination of Single and Multiple inheritance.



Inheritance

- Defining subclass:

```
class subclassname extends superclassname
{
 [variable declarations]
 [method declaration]
}
```

- '**extends**' keyword is used to deriving superclass into subclass.

Method overriding

- Method in superclass has same name and signature as a method in its subclass.
- Return type also same.

Overloading Vs Overriding

Overloading	Overriding
Method with same name and different signature	Method with same name and same signature
Done in same scope	Done in different scope
Method is called depending on number of argument or type of argument	Method is called depending on type of object
Compile time called is resolved	Runtime called is resolved

Final Keyword

- Final keyword can be used with variable, class and method.
- Final variable: Variable whose value does not change throughout program execution once it initialized.
  - Syntax:
    - final datatype variablename = value;

Final Keyword

- Final method: If we declare any method is final then we could not override that method in subclass
  - Syntax:
    - final returntype methodname (parameter list)

```
{
-----
}
```

Final Keyword

- Final class: If we declare any class is final then that class can not be inherited.
  - Syntax:
    - final class classname

```
{
-----
}
```

Final Variable → To create constant variables

Final Methods → Prevent Method Overriding

Final Classes → Prevent Inheritance

Super

- Uses
  - To call paramenterized constructor of base class in derived class
  - To call base call overridden method in derived class
  - To access base class data members in derived class. (if data members have same name in both classes)

Abstract class

- A class which is declared as abstract is known as an **abstract class**.
- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Abstract class

- Example:

```
abstract class Shape
 {
   int color;

    abstract void draw();
}
```

Abstract method

- A method which is declared as abstract and does not have implementation is known as an abstract method.
- **Example of abstract method**
- **abstract void** printStatus();//no method body and abstract

- The method body will be defined by its subclass.
- Abstract method can never be final and static.
- Any class that extends an abstract class must implement all the abstract methods.

Interface

- An interface is collection of:
  - Public abstract methods
  - Public final variables
- An interface is not class but similar like class.
- The important use of interface is to achieve abstraction.
- By interface,we can implement the **Multiple inheritance** which is not possible using only classes.

Interface

- A class describes the attribute and behaviors of object.
- A interface contains behaviors that a class implement.

Interface is similar to class

- An interface can contain any number of methods.
- An interface is written in a file with **.java** extension, with the name of the interface matching the name of the file.
- The byte code of an interface appears in **.class** file.

Interface is different to class

- You cannot instantiate an interface.
- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface cannot contain instance fields. The only fields that can appear in an interface must be declared both static and final.
- An interface is not extended by a class; it is implemented by a class.
- An interface can extend multiple interfaces.

Declaring Interface

- Syntax of Interface: interface keyword is used

```
public interface interfacename
{
 [variable declarations: public, final and static]
 [method declarations: public and abstract only]
}
```

Declaring Interface

- Example

```
public interface Area
{
 final static float pi = 3.142f;
 float compute(float x, float y);
 void show();
}
```

Implementing Interface

- Interface is used as Superclass whose properties are inherited by classes.
- Syntax:

```
class classname implements interfacename
{
 Body of class
}
```

Extending Interface

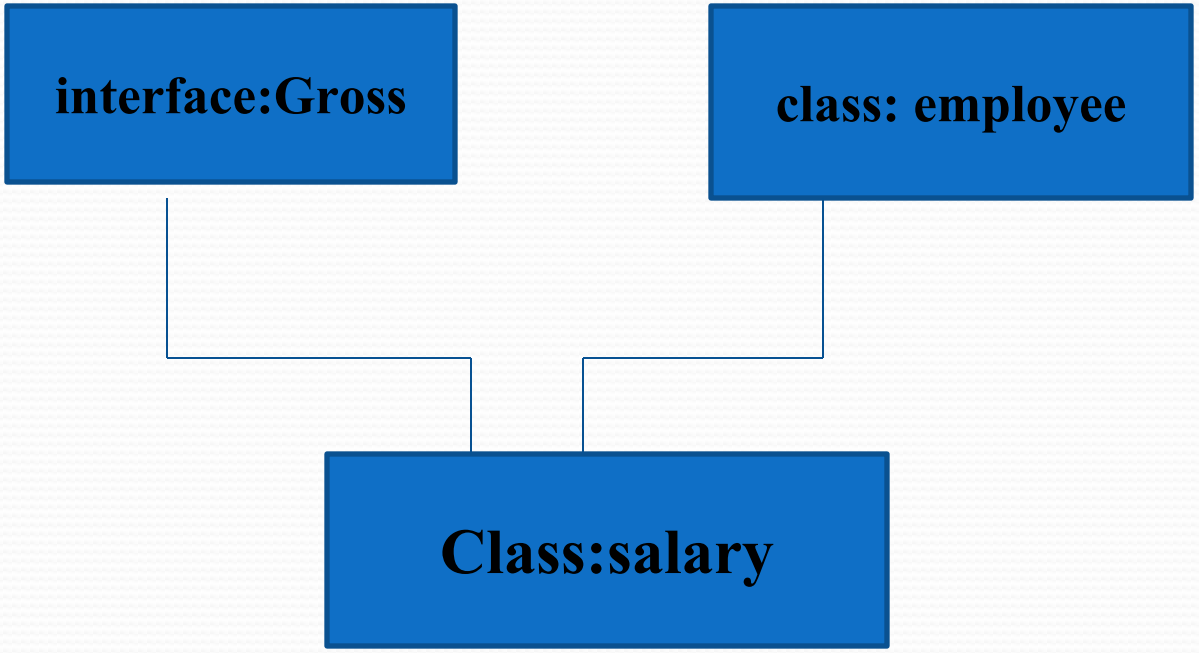
- One interface can extends from other interface like class
- Syntax:

```
interface interfacename2 extends interfacename1
{
 Declaration of variables and methods
}
```

Extending Interface

- Example

```
interface ItemConstant
{
 int code =1001; String name = “Fan”;
}
interface Item extends ItemConstants
{
 void display();
}
```


interface:Gross
class: employee
Class:salary

Interface References:

- Runtime polymorphism can be achieved using interface.
- We can create **reference** for parent interface.This reference can point to an object of the child classes which implements the corresponding interface.

Packages

- **Java provides a powerful means of grouping related classes and interfaces together in a single unit: packages.**
- **Put simply, packages are groups of related classes and interfaces.**
- **Packages provide a convenient mechanism for managing a large group of classes and interfaces while avoiding potential naming conflicts.**
- **The Java API itself is implemented as a group of packages.**

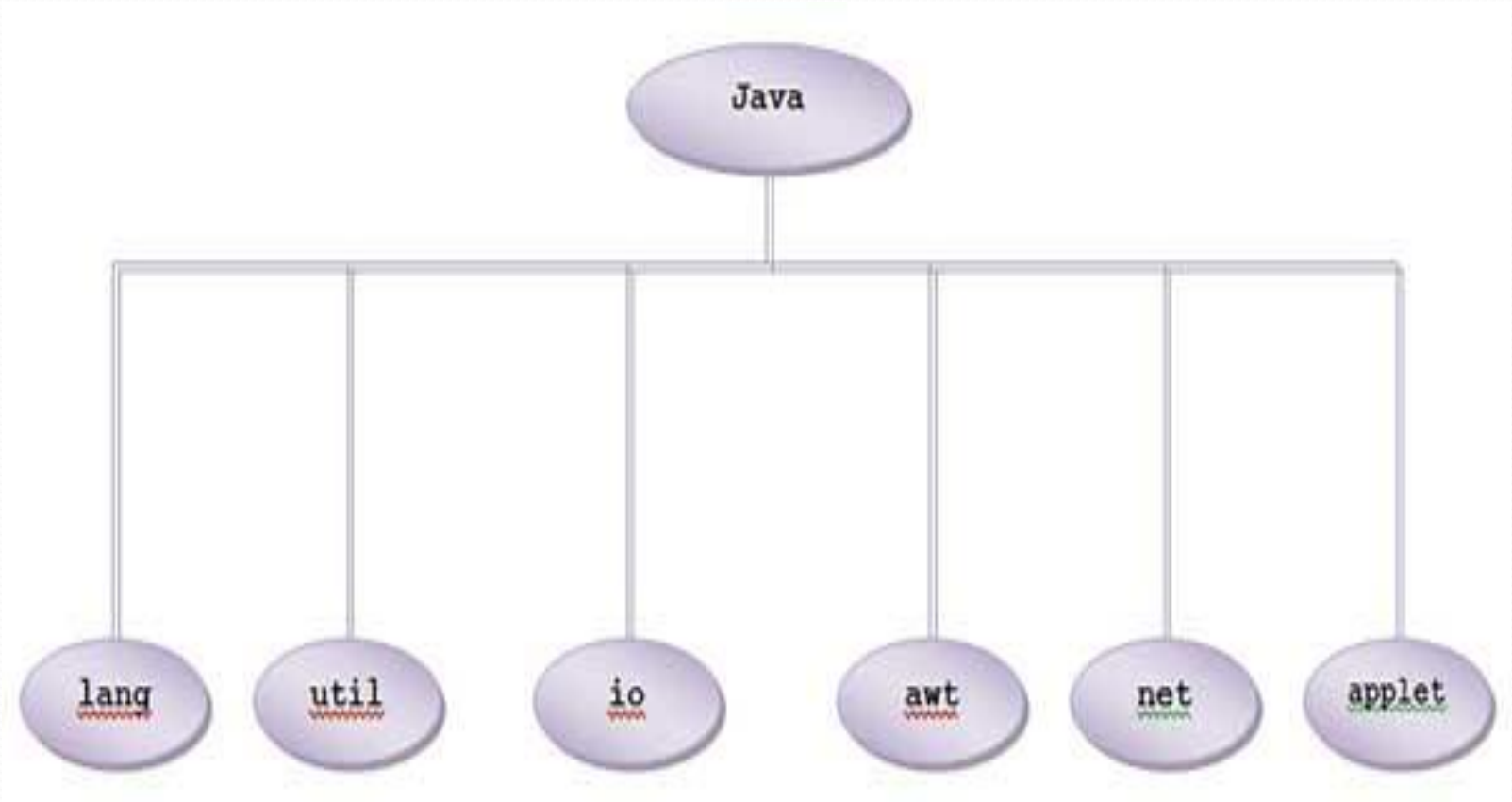
Benefits using packages

- Reusability of classes
- Packages provides a way to hide classes thus preventing other classes that meant for internal use only.
- Package provides a way for separating design from coding

Packages

- Packages are collection of:
  - Classes
  - Interfaces
- Packages are classified in :
  - Java API packages (System packages)
  - User defined packages

Java API Packages



Java API Packages:

Package name	contents
java.applet	Contains classes needed to create Java applets that run under Netscape 2.0 (or greater), HotJava, or other Java-compatible browsers.
java.awt	Contains classes helpful in writing platform-independent graphic user interface (GUI) applications. This comes with several subpackages including java.awt.peer and java.awt.image.
java.io	Contains classes for doing I/O (input and output). This is where the data stream classes are kept.
java.lang	Contains the essential Java classes. java.lang is implicitly imported, so you don't need to import its classes.
java.net	Contains the classes used for making network connections. These are used in tandem with java.io for reading and writing data from the network.
java.util	Contains other tools and data structures, such as encoding, decoding, vectors, stacks, and more. Additional packages are also available commercially.

Accessing Package

- **“import”** keyword is used to accessing package.
- Example:
  - import java.lang.Integer;
  - import java.io.BufferedReader;
  - import java.awt.Color;
  - import packagename.subpackagename.*;

Creating User Defined Packages

- Declare package at the beginning of file using:

  package packagename;
- Defined the **class** that is to be put in the package and declare it **public**.
- Create the subdirectory under the directory package where main file is stored.
- Compile file and .class file should be stored in subdirectory.

Creating Package Syntax

```
package   firstname;
public class Firstclass
{
//body of class


}
```

Hiding classes

- Hiding classes

```
 Package1 p1;
public class X
 {
 // body of X

  }
 class Y
{
// body of Y
 }
```

- Importing

```
   import p1 *;
 X  objectx;   //class X is available here
 Y  objecty;   // class y is not available here
```

Static import in Java

- In Java, static import concept is introduced in 1.5 version.
- With the help of static import, we can access the static members of a class directly without class name or any object.
- For Example: we always use sqrt() method of Math class by using Math class i.e. **Math.sqrt()**, but by using static import we can access sqrt() method directly.
- According to SUN microSystem, it will improve the code readability and enhance coding. But according to the programming experts, it will lead to confusion and not good for programming.
- If there is no specific requirement then we should **not** go for static import.

Difference between import and static import

- With the help of import, we are able to access classes and interfaces which are present in any package. But using static import, we can access all the static members (variables and methods) of a class directly without explicitly calling class name.
- The main difference is Readablity, ClassName.dataMember (System.out) is less readable when compared to dataMember(out), static import can make your program more readable

Access Modifiers(visibility control) in Java

- There are four types of Java access modifiers:
- **Private**: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
- **Default**: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
- **Protected**: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
- **Public**: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.
-

Access Modifiers(visibility control) in Java

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y